

# Distributed Database Project Proposal

**Project ID & Category:** 127 — Cloud-Native Databases (Docker & K8s)

## 1. Project Identity

- **Team Name:** Hirono
- **Team Members:** Vương Minh Thắng
- **Project Title:** Blue-Green Deployment for Schema Migration: “Zero-Downtime Update”

## 2. Objective & Problem Statement

- **The “Why”:** Giải quyết bài toán nâng cấp schema database trong môi trường production mà không gây downtime cho người dùng. Cụ thể, khi một e-commerce cần chuyển từ schema V1 (cũ) sang V2 (mới) — bao gồm thay đổi kiểu dữ liệu, tách cột, và normalize — hệ thống vẫn phải tiếp tục phục vụ requests xuyên suốt quá trình chuyển đổi.
- **Core Logic:** Chúng tôi implement Blue-Green Deployment Pattern kết hợp với Lazy Distributed Replication Protocol (Özsu & Valduriez, Ch.6). Hai cluster MySQL chạy song song với schema khác nhau (Blue = V1, Green = V2). Một Synchronization Bridge dựa trên Outbox Pattern thực hiện bidirectional async replication, sử dụng Schema Mapping Layer (Özsu & Valduriez, Ch.7) để dịch dữ liệu giữa hai schema, và Last-Write-Wins conflict resolution dựa trên timestamp.

## 3. Dataset Specification

- **Source:** DummyJSON Products API (<https://dummyjson.com/products>) hoặc Amazon Product Catalog từ Kaggle
- **Size:** ~200-500 sản phẩm, kích thước ~2-5 MB
- **Schema V1 (Blue):**
  - id BIGINT (PK)
  - title VARCHAR(255)
  - description TEXT
  - price DECIMAL(10,2)
  - discount\_percentage DECIMAL(5,2)
  - rating DECIMAL(3,2)
  - stock INT
  - category VARCHAR(50)
  - brand VARCHAR(100)

- sku VARCHAR(50)
- availability\_status VARCHAR(50)
- created\_at TIMESTAMP
- updated\_at TIMESTAMP
- **Schema V2 (Green):**
  - id VARCHAR(36) (PK)
  - title VARCHAR(255)
  - price\_amount DECIMAL(15,2)
  - discount\_percentage DECIMAL(5,2)
  - currency CHAR(3) DEFAULT 'USD'
  - rating DECIMAL(3,2)
  - stock\_quantity INT
  - category\_id BIGINT FK → bảng categories
  - brand\_id BIGINT FK → bảng brands
  - sku VARCHAR(50) UNIQUE
  - availability\_status ENUM('In Stock', 'Low Stock', 'Out of Stock')
  - dimensions JSON
  - metadata JSON
  - created\_at TIMESTAMP
  - updated\_at TIMESTAMP
- **Fragmentation Strategy:** Không phân mảnh. Đây là full replication giữa Blue và Green (theo Özsu Ch.6, không phải fragmentation theo Ch.2). Mỗi cluster lưu toàn bộ dataset nhưng dưới schema khác nhau.

#### 4. System Architecture

- **Nodes:** 3 sites
  - Blue cluster (MySQL với schema V1)
  - Green cluster (MySQL với schema V2)
  - Sync Bridge (middleware)
- **Communication Layer:** HTTP/REST (Spring Boot) giữa client và services. Sync Bridge sử dụng polling-based CDC (Change Data Capture) đọc bảng outbox\_events của mỗi cluster, kết hợp RabbitMQ làm retry queue cho failed replications.

- **Storage:** MySQL 8.0 instances chạy trong Docker containers. Mỗi instance có volume riêng để persistence. Bảng outbox\_events lưu local trong cùng database để đảm bảo atomicity với business writes (transactional outbox pattern).

## 5. Tech Stack & Implementation Plan

- **Programming Language:** Java 17
- **Deployment:** Docker Compose cho development, Kubernetes (minikube) cho final demo. K8s Service với label selector version: blue|green để traffic routing.
- **Libraries/Frameworks:**
  - Spring Boot 3 (REST API, Data JPA)
  - Liquibase (database migration)
  - MySQL Connector/J
  - RabbitMQ (message queue cho retry logic)
  - JMeter hoặc k6 (load testing)
  - Lombok (giảm boilerplate)

## 6. Success Metrics & Analysis

- **Quantitative Metric:**
  - Số transactions thành công vs thất bại trong switch-over window (deliverable bắt buộc)
  - Replication lag time của Sync Bridge: P50, P95, P99 (milliseconds)
  - Throughput (TPS) trước/trong/sau switch-over
  - Recovery time sau khi kill node (RTO)
- **The “Failure” Scenario:**
  - Kill Green pod giữa switch-over bằng kubectl delete pod — kiểm tra K8s tự reschedule, không mất transaction
  - Kill Sync Bridge giữa chừng — kiểm tra outbox events được pickup khi service restart
  - Network partition giữa Blue và Sync Bridge — kiểm tra retry queue
  - Concurrent conflict: cùng product được update đồng thời trên cả Blue và Green — kiểm tra Last-Write-Wins resolution
  - Rollback scenario: phát hiện V2 lỗi, switch ngược về Blue — kiểm tra Sync Bridge đảo chiều

## 7. Project Milestones

- **Milestone 1 (Ngày 1-5):** Docker Compose chạy 2 MySQL + Liquibase migrate schema V1, V2 hoàn tất. Blue service và Green service CRUD API hoạt động độc lập.
- **Milestone 2 (Ngày 6-12):** Sync Bridge hoàn chỉnh: Outbox poller + Schema translator V1↔V2 + Conflict resolver. Test end-to-end: write vào Blue → tự động xuất hiện ở Green với schema đã translate.
- **Milestone 3:** Deploy lên Kubernetes. Hoàn tất 5 failure scenarios. Quay video demo. Viết Analysis Report. Submit.